

Date: 2026-09-21

Topic: Day 4 - Generative AI & LLMs

What I learned:

- How to connect to an LLM API using the Python OpenAI SDK.
- Understanding of core concepts: Tokens, Context Window, and Temperature.
- How to structure system and user prompts to guide the AI's behavior and reduce hallucination.
- Techniques to force an LLM to output structured data (JSON).
- How string formatting in Python interacts with JSON braces, and how to escape them.

```
[www] PS C:\Wilson\AI-engineer-Internship\github\AI-engineer-training\AI-engineer-training\week-03\Day-04\Practical_Task> python app.py

AI TEXT ASSISTANT
=====

Select an operation:
1. Question Answering
2. Summarization
3. Rewriting
4. Exit

Enter your choice: 1

Enter your question:
What is Generative AI?

--- Answer ---
**Generative AI** (Generative Artificial Intelligence) is a type of artificial intelligence that can create new content such as text, images, video, audio, code, and 3D models based on patterns it learned from existing data.

Here is a breakdown of how it works, what it does, and how it differs from traditional AI:

---

### How it works
1. **Training on Data:** Generative AI models are trained on massive datasets (such as billions of pages of text or millions of images).
2. **Learning Patterns:** Rather than memorizing the data, the model learns the underlying patterns, structures, and relationships within it (for example, how words fit together in a sentence or how light and shadow work in an image).
3. **Generating Output:** When a user provides a prompt (an instruction or question), the model uses these learned patterns to predict and generate a brand-new response.

---

### Key Capabilities and Examples
* **Text Generation:** Writing essays, summarizing long documents, translating languages, or answering questions (e.g., "ChatGPT, Claude, Gemini").
* **Image Generation:** Creating digital artwork, realistic photographs, or graphics from text descriptions.
```

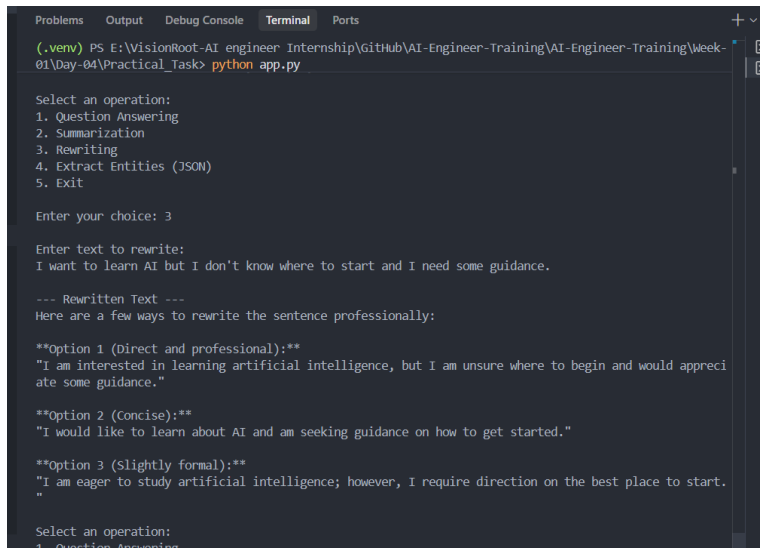
```
Select an operation:
1. Question Answering
2. Summarization
3. Rewriting
4. Extract Entities (JSON)
5. Exit

Enter your choice: 2

Enter text to summarize:
Machine learning is a branch of artificial intelligence that allows computers to learn patterns from data. Instead of explicitly programming every rule, machine learning algorithms use historical data to build models that can make predictions or decisions on new data.

--- Summary ---
**Summary:**

Machine learning is a branch of artificial intelligence where computers use past data to learn patterns and make predictions or decisions, without needing to be programmed for every specific rule.
```



```
(.venv) PS E:\VisionRoot-AI engineer Internship\Github\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-04\Practical_Task> python app.py

Select an operation:
1. Question Answering
2. Summarization
3. Rewriting
4. Extract Entities (JSON)
5. Exit

Enter your choice: 3

Enter text to rewrite:
I want to learn AI but I don't know where to start and I need some guidance.

--- Rewritten Text ---
Here are a few ways to rewrite the sentence professionally:

**Option 1 (Direct and professional):**
"I am interested in learning artificial intelligence, but I am unsure where to begin and would appreciate some guidance."

**Option 2 (Concise):**
"I would like to learn about AI and am seeking guidance on how to get started."

**Option 3 (Slightly formal):**
"I am eager to study artificial intelligence; however, I require direction on the best place to start."

Select an operation:
1. Question Answering
```

What I implemented:

- An AI text assistant CLI application in Python.
- Features for Question Answering, Text Summarization, and Text Rewriting.
- A new feature to extract entities (names, places, organizations, dates) from text and return them as a structured JSON response.

What worked:

- Successfully formatting system prompts to enforce specific rules (e.g., no hallucination, specific JSON format).
- Routing requests through the OpenAI SDK to a free alternative model (Google Gemini) by updating the `base\_url`.

What failed:

- The `requirements.txt` file was improperly formatted (used spaces instead of `==`), preventing dependency installation.
- The original OpenAI API key returned a `429 Insufficient Quota` error due to zero billing credits on the account.

- When generating the structured JSON response, Python threw a 'KeyError: '\n"entities"'.

How I solved it:

- Activated the correct local 'venv' for the Day-04 project and re-formatted 'requirements.txt' to correctly specify package versions with '=='.
- Migrated the application to use Google Gemini's free API by replacing the OpenAI API key with a Gemini key and updating the 'base_url' to Google's OpenAI-compatible endpoint.
- Fixed the 'KeyError' by escaping the JSON curly brackets using double brackets '{{ }}' in the prompt string, preventing Python's '.format()' method from misinterpreting them as variables.

What I would improve:

- Add error handling for JSON decoding in case the model returns malformed text.
- Implement streaming responses so the user doesn't have to wait for the entire text to generate before reading it.